

7.15 工作日志

一、内容概述

主要工作围绕环境搭建、项目运行和团队协作展开。具体包括：完成前端、后端、数据库环境的安装与配置；安装相关 IDE（VS Code）并测试 Demo 程序成功运行；组长为小组创建 GitHub 项目仓库，小组成员共同在项目内协作，明确分工；学习并使用 Git 常用操作（clone、commit、push、pull、branch 等）。

二、GitHub 项目创建与团队协作

2.1 GitHub 仓库创建

组长在 GitHub 上为小组创建了统一的项目仓库（campus-practice），所有小组成员作为 Collaborator 加入，共同在一个项目内进行协作开发。仓库创建成功截图如下：

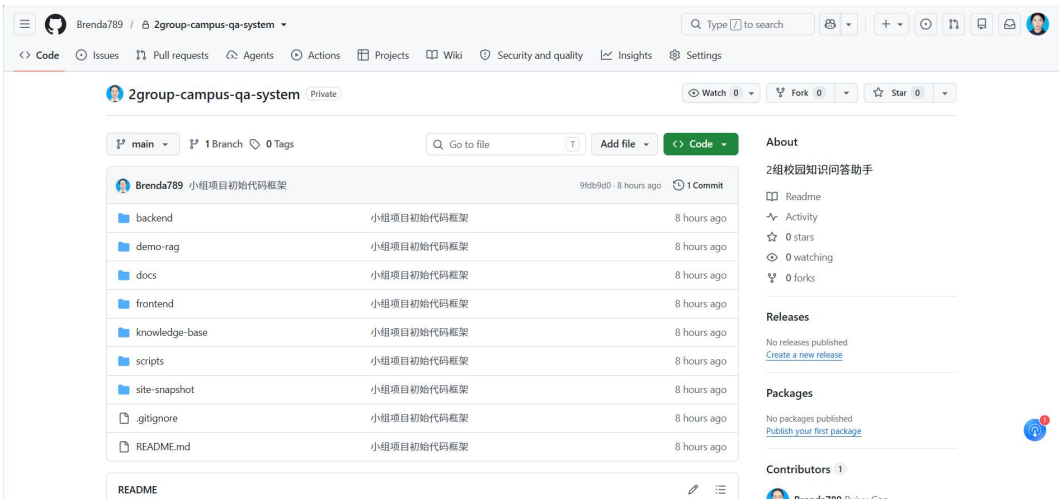


图 1 GitHub 仓库创建成功截图

2.2 Git 常用操作学习

今日学习和使用了以下 Git 常用操作命令：

操作	命令	说明
克隆仓库	git clone <url>	将远程仓库代码克隆到本地
查看状态	git status	查看当前工作区和暂存区状态
添加文件	git add <file>	将文件添加到暂存区
提交更改	git commit -m "message"	将暂存区内容提交到本地仓库
推送到远程	git push origin <branch>	将本地提交推送到远程仓库
拉取更新	git pull origin <branch>	从远程仓库拉取最新代码

创建分支	<code>git checkout -b <branch></code>	创建并切换到新分支
切换分支	<code>git checkout <branch></code>	切换到已有分支
查看分支	<code>git branch</code>	列出所有本地分支
查看日志	<code>git log --oneline</code>	查看提交历史记录

三、系统架构设计

本系统采用前后端分离架构，前端基于 Vite 8 + React 19 + TypeScript 构建（Ant Design 6 + Tailwind CSS 4），后端基于 Spring Boot（Java 17 + Maven），数据库使用 MySQL，AI 能力通过阿里云 DashScope（百炼）API 提供。系统整体架构如下图所示：



图 2 系统架构设计图

架构说明：系统分为四层。表现层包括运行在 8080 端口的门户首页（含问答机器人浮窗）和运行在 5173 端口的 React 19 管理后台（TypeScript + Ant Design 6 + Tailwind CSS 4），负责收集用户提问并展示答案，不处理复杂计算；业务逻辑层基于 Spring Boot（Java 17 + Maven）构建 RESTful API 服务，运行在 8000 端口，作为系统核心调度中心，集成 LlamaIndex 实现 RAG（检索增强生成）问答，

负责决定从本地检索还是调用外部大模型，并将结果拼接后返回前端；数据层使用 MySQL 关系型数据库（端口 3306，存储管理员账号、历史聊天记录等业务数据）和 FAISS 向量索引（llama_index_storage 目录，存放校园指南文档的语义向量，支持高效相似度检索）；外部服务层通过 HTTPS 调用阿里云 DashScope API（百炼大模型），当后端无法直接回答时，打包数据发送给大模型推理并取回结果。前端通过 Vite 8 代理将 API 请求（Axios）转发至后端 8000 端口，三端（8080、5173、8000）协同工作，共同构成完整的校园知识问答系统。

四、环境安装过程

4.1 系统环境要求

操作系统：Windows 10 1809+ 或 Windows 11；内存要求：8GB 以上。项目放置在 D 盘根目录，路径中不含中文字符，使用管理员权限运行 PowerShell。

4.2 一键安装开发工具

使用管理员 PowerShell 执行一键安装脚本，配置国内镜像源加速下载。安装工具包括：Git、Node.js 20 LTS、Python 3.12、uv 包管理器。安装成功截图如下：

```
PS C:\WINDOWS\system32> git --version
>> node -v
>> npm -v
>> python --version
>> uv --version
git version 2.49.0.windows.1
v24.18.0
11.16.0
Python 3.11.0
uv 0.11.28 (ebf0f43d7 2026-07-07 x86_64-pc-windows-msvc)
```

图 3 一键安装开发工具脚本执行截图

安装完成后关闭 PowerShell 并重新以管理员身份打开，验证各工具版本：

工具	验证命令	结果
Git	git --version	git version 2.49.0
Node.js	node -v	v24.18.0
npm	npm -v	11.16.0
Python	python --version	Python 3.11.0
uv	uv --version	uv 0.11.28

4.3 后端环境安装

克隆项目后进入 backend 目录，使用 uv sync --extra dev 安装后端 Python 依赖。使用 copy .env.example .env 命令复制环境变量模板，编辑 .env 文件填入阿里云 DashScope API Key。

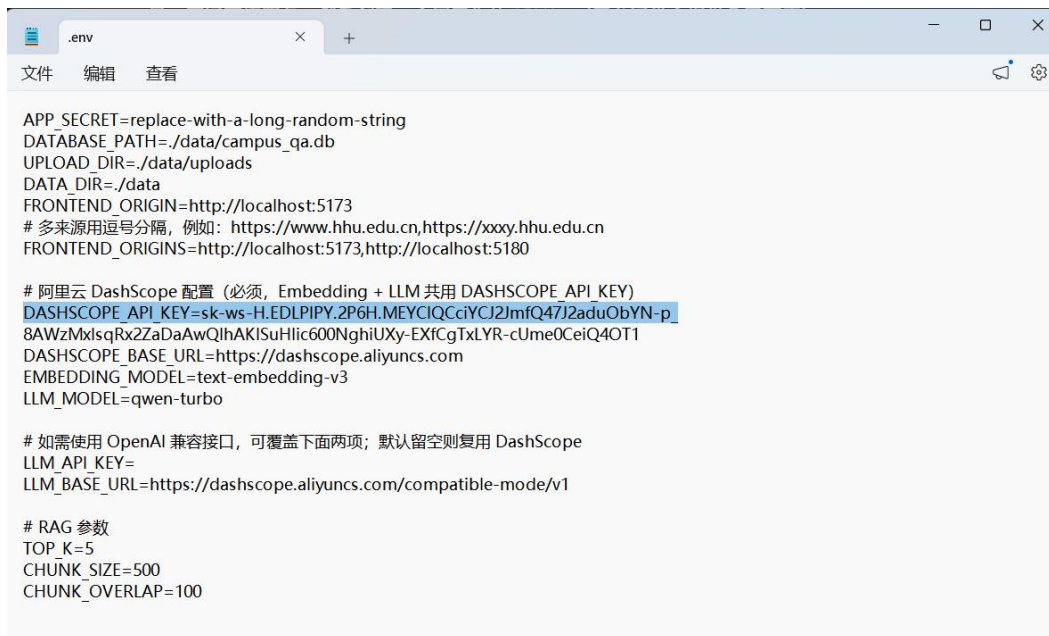


图 4 编辑.env文件配置 API Key

执行 `uv run uvicorn app.main:app --reload` 启动后端服务, 服务运行在 8000 端口, 可通过 `http://localhost:8000/docs` 访问 API 文档。后端成功运行截图如下:

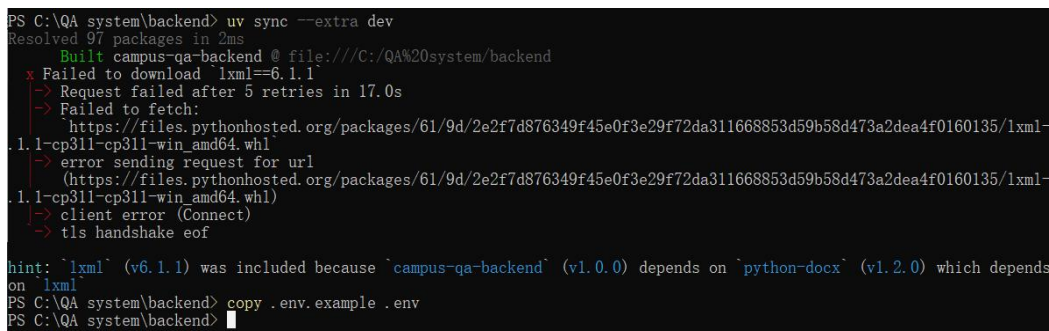


图 5 后端服务成功运行截图

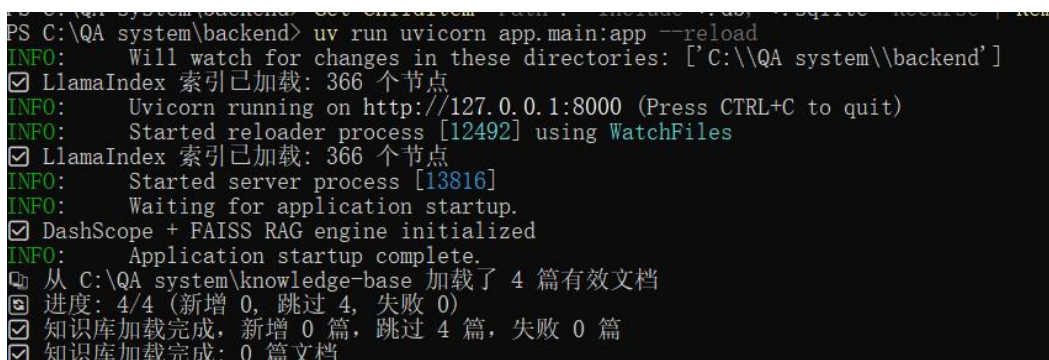


图 6 后端服务端口 8000 运行状态

4.4 前端环境安装

进入 frontend 目录执行 npm install 安装前端依赖，使用 npmmirror 国内镜像加速。执行 npm run dev 启动 Vite 开发服务器，运行在 5173 端口。前端运行截图如下：

```
PS C:\QA system> cd frontend
PS C:\QA system\frontend> npm install

up to date in 3s

15 packages are looking for funding
  run 'npm fund' for details
```

图 7 前端管理后台启动成功截图

```
PS C:\WINDOWS\system32> cd "C:\QA system\frontend"
>> npm run dev

> frontend@0.0.0 dev
> vite

20:07:43 [vite] (client) Re-optimizing dependencies because vite config has changed

VITE v8.1.2 ready in 780 ms
  ➔ Local:   http://localhost:5173/
  ➔ Network: use --host to expose
  ➔ press h + enter to show help
20:07:44 [vite] (client) [optimizer] bundling dependencies...
```

图 8 前端管理后台端口 5173 运行状态

4.5 门户首页部署

进入 site-snapshot 目录，使用 py -m http.server 8080 启动静态服务器，提供门户首页和问答机器人 Widget 访问。

```
PS C:\QA system\site-snapshot> py -m http.server 8080
Serving HTTP on :: port 8080 (http://[::]:8080/) ...
```

图 9 静态页面端口 8080 运行状态

五、项目运行效果展示

三个终端服务全部启动成功后，通过浏览器分别访问各页面，验证系统功能正常。

5.1 前端页面



图 10 前端页面整体效果

5.2 后端页面



图 11 门户首页效果



图 12 管理员后台页面效果 1



图 13 管理员后台页面效果 2

5.3 API 文档页面

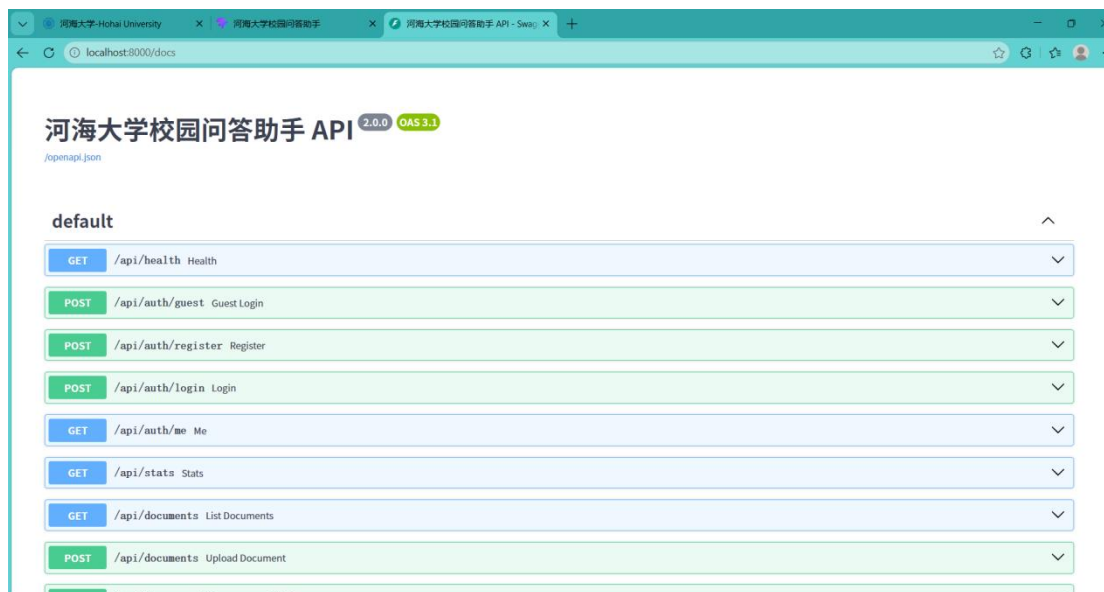


图 14 Swagger API 文档页面（自动生成）

六、遇到的困难及解决方案

在环境搭建和项目运行过程中，遇到了以下六个主要问题，经过团队协作和排查均已成功解决：

1. GitHub 仓库克隆失败

问题描述：在使用 `git clone` 命令克隆 GitHub 仓库时，由于网络原因，无法直接克隆 GitHub 上的项目代码，出现连接超时和下载速度极慢的问题。

解决方案：通过团队成员之间共享项目压缩包的方式，由已经成功克隆的成员将项目文件夹压缩后，通过 U 盘或局域网共享给其他成员，解压后直接使用。后续配置好国内镜像源后即可正常进行依赖安装。

2. uv sync 官方源连接超时

问题描述：执行 `uv sync` 命令安装 Python 依赖时，出现 "Failed to fetch" 和 "tls handshake eof" 错误，无法从 Python 官方源获取依赖包，导致后端环境无法搭建。

解决方案：通过设置环境变量 `UV_INDEX_URL` 指向清华大学镜像源（<https://pypi.tuna.tsinghua.edu.cn/simple>），或使用 `--index-url` 参数指定镜像源，实现了稳定快速的依赖下载。同时删除可能残留的 `.venv` 目录后重新安装，确保环境干净。

3. 数据库初始化失败

问题描述：执行 `uv run` 启动后端服务时，终端报错 `sqlite3.IntegrityError: UNIQUE constraint failed: users.email`。这是因为之前残留的数据库文件中存在重复的初始化数据，导致重新运行时违反了唯一性约束。

解决方案：使用 PowerShell 命令强制删除残留的数据库文件：`Get-ChildItem -Path . -Include *.db, *.sqlite -Recurse | Remove-Item -Force`。删除后重新启动后端服务，系统自动创建全新的数据库并成功初始化种子数据。

4.静态页面启动失败——8080 端口被占用

问题描述：在 `site-snapshot` 目录执行 `py -m http.server 8080` 启动静态页面服务器时，终端报错`[WinError 10013]`，提示没有权限绑定 8080 端口，原因是该端口已被其他进程占用。

解决方案：首先使用 `netstat -ano | findstr :8080` 定位占用端口的进程 PID（为 6208），然后执行 `taskkill /PID 6208 /F` 强制终止该进程释放端口。释放 8080 端口后重新执行启动命令，静态页面服务正常运行。

5.管理后台登录失败——401 认证错误

问题描述：前后端服务均正常运行，但在管理后台登录页面使用文档提供的默认账号（`admin@campus.example / admin123`）登录时失败，显示 `POST /api/auth/login` 返回 `401 Unauthorized`。

解决方案：通过检查后端 `main.py` 中的 `seed_database` 函数，确认初始化管理员账号的实际密码与文档记录有出入。以源代码中实际写入数据库的账号密码为准，成功登录后台管理系统。

6.前端接口请求被拒——ECONNREFUSED

问题描述：在管理后台（Vite 开发服务器控制台）中频繁出现`[vite] http proxy error: /api/conversations AggregateError [ECONNREFUSED]`错误，前端发起的 API 代理请求被后端拒绝连接。

解决方案：排查发现是因为在某个终端关闭后，后端服务（8000 端口）意外停止运行。确保三个终端（后端 8000、前端 5173、静态页 8080）全部保持运行状态，不要关闭任何终端窗口。重新启动后端服务后刷新页面，前后端通信恢复正常。

今日问题汇总如下表：

序号	问题	原因	状态
1	GitHub 仓库克隆失败	校园网络限制外网访问	已解决

2	uv sync 官方源连接超时	PyPI 官方源国内访问不稳定	已解决
3	数据库初始化唯一约束冲突	残留数据库文件数据重复	已解决
4	8080 端口被占用	端口被其他进程占用	已解决
5	管理后台 401 认证失败	文档密码与实际代码不一致	已解决
6	前端 ECONNREFUSED 代理错误	后端服务意外停止运行	已解决

七、总结

今日顺利完成了校园知识问答助手项目的全部环境搭建工作。前端、后端和数据库三层环境均成功安装并验证通过，Demo 程序能够在本地完整运行。团队成员掌握了 Git 的基本操作，建立了 GitHub 协作仓库，明确了各自分工。在搭建过程中遇到了网络、端口、数据库等方面的问题，通过团队协作均成功排查和解决，积累了宝贵的实战经验。

(1) 掌握了 Windows 环境下全栈开发环境的搭建方法。通过本次实践，学会了使用 winget 一键安装 Git、Node.js、Python 等开发工具，理解了前端（Vite 8 + React 19 + TypeScript）、后端（Java 17 + Maven）和数据库（MySQL）三层架构的协作关系，为后续项目开发打下了坚实的基础。

(2) 学会了配置国内镜像源解决依赖安装问题。在使用 uv（Python 包管理器）和 npm（Node.js 包管理器）时，通过切换至清华大学镜像源和 npmmirror 镜像源，显著提升了下载速度和稳定性。

(3) 掌握了多终端协同运行项目的技巧。项目需要同时启动三个服务（后端 8000、前端 5173、静态页 8080），学会了使用 netstat、taskkill 等 Windows 命令行工具排查端口占用问题，提高了故障排查能力。

(4) 系统学习了 Git 版本控制工具的核心操作。从克隆仓库（clone）、添加文件（add）、提交更改（commit）、推送到远程（push）到拉取更新（pull），形成了完整的 Git 工作流认知。

(5) 学会了查阅终端错误日志定位问题根因。通过仔细阅读报错信息（如 IntegrityError、ECONNREFUSED、401 Unauthorized 等），能够快速定位问题所在模块，有针对性地进行修复。